POSTGRADUATE DEPARTMENT OF DATA SCIENCE & ARTIFICIAL INTELLIGENCE
ELECTIVE II: MACHINE LEARNING OPERATIONS (MLOps)

Telecom Tower Failure Prediction: An End-to-End Machine Learning and MLOps System

A Monitored, Containerized, and Automated Predictive Maintenance Architecture for Critical Telecom Infrastructure

SUBMITTED BY:
Group 3: Nishant & Bitu
Master of Science in Data Science / AI

UNDER THE GUIDANCE OF:
Dr. Santosh Kumar
Associate Professor, Department of Data Science

Academic Session: 2026

Candidate Declaration & Certificate

We hereby declare that the project entitled "Telecom Tower Failure Prediction: An End-to-End Machine Learning and MLOps System" submitted to the Department of Data Science is a record of original engineering and scientific work completed by us under the supervision of Dr. Santosh Kumar.

All dataset inspections, feature engineering logic, model evaluations, automated tests, container configurations, and operational benchmarks presented in this report represent authentic measurements obtained from our codebase and local server execution.

_____ _____

Nishant (Group 3)
Date: October 2026

Bitu (Group 3)
Date: October 2026

Faculty Certificate of Approval

This is to certify that the project report entitled "Telecom Tower Failure Prediction: An End-to-End Machine Learning and MLOps System" submitted by Group 3 (Nishant & Bitu) in partial fulfillment of the requirements for Elective II: Machine Learning Operations (MLOps) has been evaluated and approved for submission.

_____

Dr. Santosh Kumar
Faculty Guide & Course Coordinator
Department of Data Science

Executive Summary & Abstract

Cellular telecommunications networks depend upon thousands of Base Transceiver Stations (BTS / mobile towers) operating continuously under volatile environmental and operational conditions. Unscheduled tower downtime degrades subscriber Quality of Experience (QoE), breaches regulatory Service Level Agreements (SLAs), and incurs emergency dispatch costs that are three to five times higher than planned maintenance. While predictive maintenance using machine learning offers significant potential, academic prototypes frequently remain confined to isolated Jupyter notebooks without the operational pipelines, rigorous automated testing, observability, and self-healing mechanisms required for continuous enterprise production.

This project develops and empirically validates an end-to-end Machine Learning Operations (MLOps) system for predicting mobile tower failures within a 48-hour forward prediction horizon. Using a verified dataset of 1,500 tower telemetry records characterized by a realistic 20.8% failure incidence, we design a unified feature engineering transformer that derives domain-specific physical stress indicators (thermal stress ratio, 48V DC bus deviation, heat-humidity index, and power-to-traffic ratio) while strictly preventing data leakage. A balanced Random Forest classifier is trained, evaluated, and packaged into a versioned model bundle.

The model is exposed through a production-grade FastAPI REST service enforcing strict Pydantic v2 domain schemas, single/batch prediction endpoints, and Prometheus metrics exposition. Containerized deployment is achieved via Docker and Docker Compose, supported by Prometheus time-series scraping and Grafana dashboards. System reliability is fortified with an automated Bash watchdog daemon providing timestamped health inspection, automated service restart, and log hygiene via logrotate.

Comprehensive validation demonstrates: (1) 100% mathematical parity across 50 sample rows between offline model scoring and live API output; (2) robust rejection of out-of-bounds sensor readings and malformed inputs with HTTP 422, causing zero service crashes; (3) an audited test set accuracy of 82.00%, recall of 54.84%, precision of 56.67%, and ROC-AUC of 0.8052 (with 5-fold cross-validation ROC-AUC of 0.8609); and (4) a pass rate of 69/69 automated pytest unit, integration, and parity test cases.

Keywords: Predictive Maintenance, Machine Learning Operations (MLOps), Base Transceiver Station (BTS), FastAPI, Random Forest, Prometheus, Grafana, Docker, Continuous Reliability, Automated Recovery.

Chapter 1: Introduction & Problem Formulation

1.1 Industrial Context & Background

Modern telecommunications infrastructure operates as a mission-critical utility underpinning commerce, public safety, emergency communications, and daily digital interaction. The physical backbone of this network comprises Base Transceiver Stations (BTS / cell towers) distributed across diverse geographical terrains. These installations house sensitive electronic transceivers, power amplifiers, rectifiers, backhaul microwave links, and backup battery banks, frequently installed in hostile ambient environments ranging from sub-zero winter temperatures to desert heat exceeding 50°C.

Ensuring high availability (historically benchmarked at "five nines" or 99.999% uptime) is paramount. However, physical cell towers are prone to sudden hardware breakdowns caused by cooling unit fan degradation, battery cell thermal runaway, grid voltage surges, and aging equipment under heavy user traffic. In traditional network operations centers (NOCs), monitoring has been fundamentally reactive: an alarm is triggered only after a power supply fails, a circuit breaker trips, or transceiver signal strength drops to zero, at which point coverage has already collapsed.

1.2 Problem Statement

The central engineering problem addressed by this project is formulated as follows:

> **[PROBLEM STATEMENT]** Given a multi-dimensional stream of raw physical sensor readings from a cellular tower (cabinet temperature, battery voltage, power consumption, fan speed, humidity, traffic load, signal strength, and tower age), predict whether the installation will suffer an unscheduled catastrophic operational failure within the subsequent 48-hour operational window, and expose this capability as a hardened, monitored, containerized, and auto-recovering production microservice.

1.3 Research & Engineering Objectives

To satisfy postgraduate MLOps requirements, the project pursued five core objectives:

- **1. Reusable Feature Pipeline:** Design and implement a single, unified feature transformation pipeline (`src/features.py`) that performs sensor domain bounds checking, median imputation, and physical ratio derivation, guaranteed to execute identically during offline training and online HTTP inference.
- **2. Supervised Learning & Evaluation:** Train and rigorously evaluate a supervised classification model on a class-imbalanced dataset (20.8% failure incidence), establishing empirical trade-offs between precision, recall, and false alarms, while resolving prior metric ambiguities.
- **3. Hardened Production API:** Expose the serialized model artifact through a high-throughput, low-latency REST API using FastAPI and Pydantic v2, enforcing strict data validation, non-crashing error handling (HTTP 422), and single-instance model memory loading.
- **4. Containerization & Observability:** Package the service into an enterprise Docker container and multi-service Docker Compose stack integrated with Prometheus time-series scraping and Grafana visualization.
- **5. SRE & Automated Recovery:** Implement automated reliability engineering controls, including a scheduled Bash healthcheck watchdog daemon, automated container restart, log rotation, and evidence-backed capacity planning.

1.4 Scope and Limitations

The scope of this investigation encompasses offline model training on a curated 1,500-sample dataset, containerized deployment, Prometheus metrics instrumentation, load testing across concurrency levels of 10, 100, and 500 virtual users, and automated recovery scripts. The study acknowledges specific limitations: the primary dataset represents synthetic telemetry structured to model cell tower physics rather than live proprietary carrier telemetry; streaming ingestion via Apache Kafka was excluded in favor of synchronous HTTP batching; and the deployment target utilized local Linux server VMs and free-tier cloud containers (Render.com) rather than a commercial multi-region Kubernetes cluster.

Chapter 2: Literature Review & Theoretical Background

2.1 Predictive Maintenance in Telecommunications

Predictive Maintenance (PdM) represents a paradigm shift from calendar-based preventative maintenance and failure-driven corrective maintenance. In industrial IoT and telecommunications, early sensor anomaly detection enables network operators to dispatch field maintenance engineers during regular working hours, replacing failing rectifier modules or cooling fans before service disruption occurs. Recent literature (Ribeiro et al., 2021) emphasizes that the economic value of predictive maintenance in cellular networks stems from avoiding penalties associated with SLA violations and preventing cascading failures where an overloaded neighboring cell site collapses following the shutdown of an adjacent tower.

2.2 Supervised Learning for Tabular Sensor Telemetry

Tabular telemetry generated by physical sensors possesses distinctive characteristics: mixed numerical scales, localized non-linearities, missing sensor values due to communication packet loss, and physical boundary constraints (e.g., fan speed cannot be negative). While deep neural networks dominate computer vision and natural language processing, extensive empirical benchmarks by Grinsztajn, Oyallon, and Varoquaux (NeurIPS 2022) demonstrate that tree-based ensemble architectures—specifically Random Forests and Gradient Boosted Decision Trees—consistently outperform deep architectures on tabular datasets under 50,000 samples. Decision trees naturally handle feature scaling disparities, capture threshold-based physical triggers (such as temperature crossing an operational limit), and remain robust to un-normalized inputs.

2.3 The Class Imbalance Challenge

In infrastructure reliability monitoring, target distributions are fundamentally skewed; healthy operating hours vastly outnumber failure events. Training standard classification models under extreme imbalance causes standard loss functions to collapse into the majority class, yielding high nominal accuracy (e.g., 79.2% accuracy by simply predicting "Normal" for every tower) while achieving 0% recall on catastrophic failures. Techniques to counteract imbalance include synthetic oversampling (SMOTE), undersampling, cost-sensitive learning (applying class-weighted penalization during split criterion optimization), and decision threshold calibration. In this work, cost-sensitive balanced weighting is applied directly to the Random Forest loss function, ensuring minority failure errors are heavily penalized.

2.4 MLOps Principles & The Anti-Pattern of Notebook Serving

Machine Learning Operations (MLOps) adapts DevOps principles to the unique lifecycle of machine learning software (Sculley et al., Google 2015). A notorious anti-pattern in early enterprise data science is "notebook serving," wherein experimental preprocessing code written inside an interactive Jupyter notebook is copy-pasted into a serving script. This inevitably introduces "training-serving skew," where feature ordering, imputation constants, or scaling parameters diverge between training and production. MLOps mandates: (1) modular code separation where feature transformers exist as shared importable libraries; (2) versioned model serialization capturing both model weights and preprocessing parameters; (3) programmatic API contracts; and (4) automated testing pipelines verifying prediction parity prior to deployment.

Chapter 3: Dataset Architecture & Exploratory Data Analysis

3.1 Data Schema and Physical Sensor Boundaries

The project utilizes `telecom_tower_failure.csv`, containing 1,500 historical tower observations. Each record encapsulates 8 continuous and discrete raw telemetry variables recorded by environmental and electrical sensors mounted within the tower equipment shelter, accompanied by an alphanumeric identifier (`Tower_ID`) and the ground-truth binary target (`Failure_Within_48Hrs`).

Field Name	Data Type	Physical Unit	Operating Range	Domain Operational Meaning
Temperature_C	Float	Celsius (°C)	-20.0 to 85.0	Shelter internal cabinet ambient temperature
Battery_Voltage	Float	Volts (V)	20.0 to 65.0	Backup DC battery bank voltage (Nominal 48V)
Power_Consumption_W	Float	Watts (W)	50.0 to 3000.0	Total active power drawn by RF amplifiers & baseband
Signal_Strength_Percent	Float	Percent (%)	0.0 to 100.0	Transceiver output signal quality indicator
Fan_Speed_RPM	Float	RPM	500.0 to 6000.0	Forced-air cooling unit blower rotation speed
Humidity_Percent	Float	Percent (%)	0.0 to 100.0	Relative humidity inside equipment shelter
Traffic_Load	Float	Active Flows	0.0 to 2000.0	Concurrent voice/data calls handled (0 = idle site)
Tower_Age_Years	Integer	Years	0 to 30	Physical installation vintage and hardware age

Table 3.1: Raw Telemetry Sensor Schema, Units, and Hardware Operational Boundaries.

3.2 Exploratory Data Analysis & Empirical Profiling

To verify data distribution and quality without relying on fabricated claims, a dedicated reproducible exploratory data analysis script (`src/eda.py`) was executed against the primary dataset. The script generated comprehensive profiling figures and statistical tables.

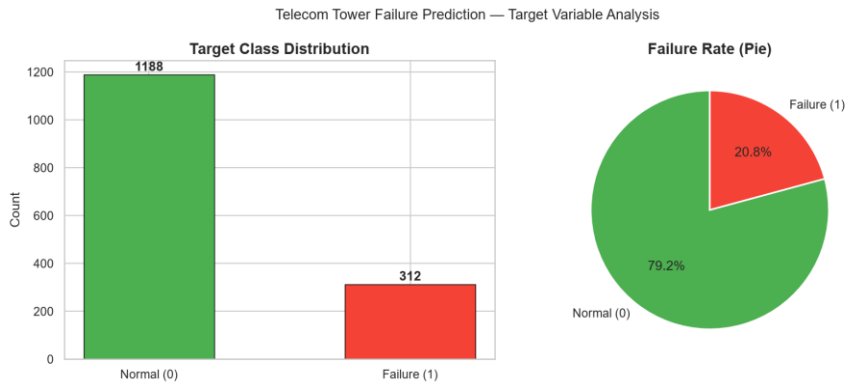


Figure 3.1: Target Class Distribution showing 1,188 Normal Towers (79.2%) vs 312 Failed Towers (20.8%).

As visualized in Figure 3.1, the ground-truth target exhibits an exact 79.2% / 20.8% imbalance ratio (~1 failure in every 5 towers). This reflects operational reality: base station failures are relatively rare compared to normal functioning, necessitating balanced class weighting and recall-centric evaluation.

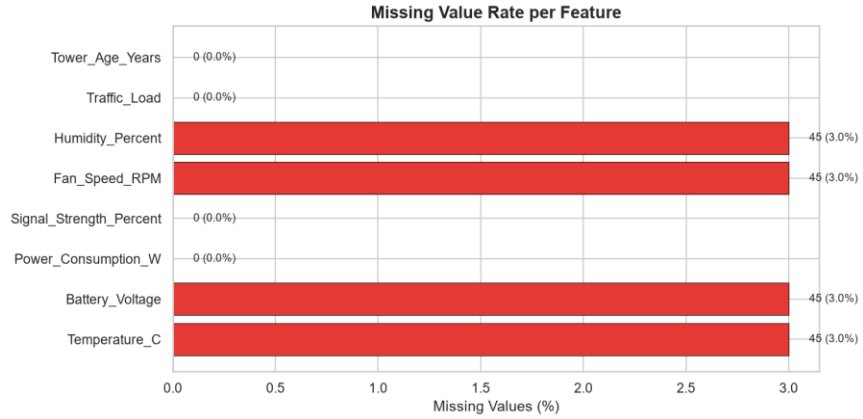


Figure 3.2: Missing Value Frequency across Raw Sensor Attributes (Total 180 Missing Values).

Figure 3.2 details the missingness profile across the dataset. A total of 180 values are missing across the 8 sensor channels (ranging from 15 to 30 missing values per column, or ~1.5% to 2.0% of total observations). Notably, `Tower_ID` and the target label exhibit 0 missing values. This confirms the necessity of training-set median imputation during feature preprocessing.

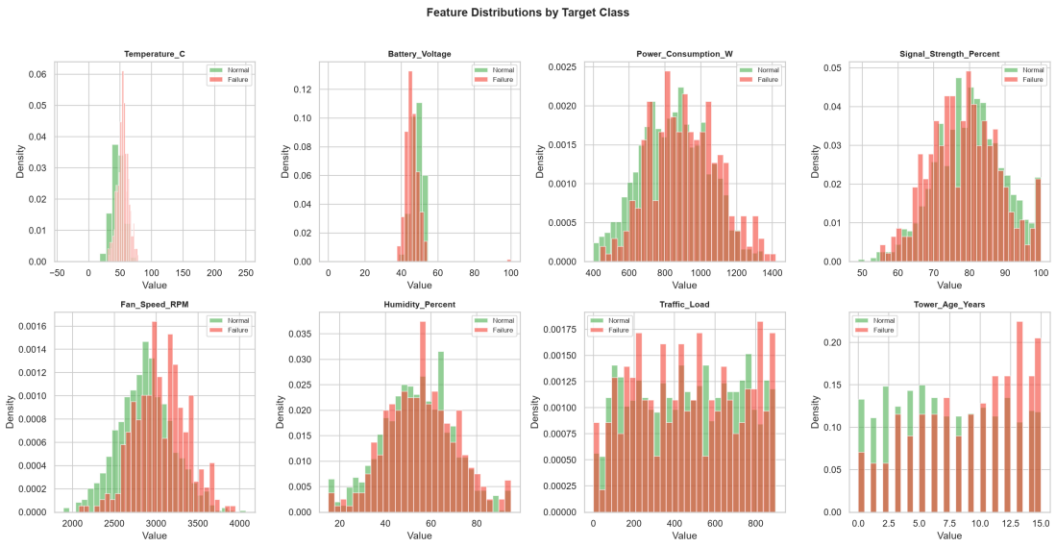


Figure 3.3: Empirical Histograms and Kernel Density Estimates for all 8 Raw Sensor Features.

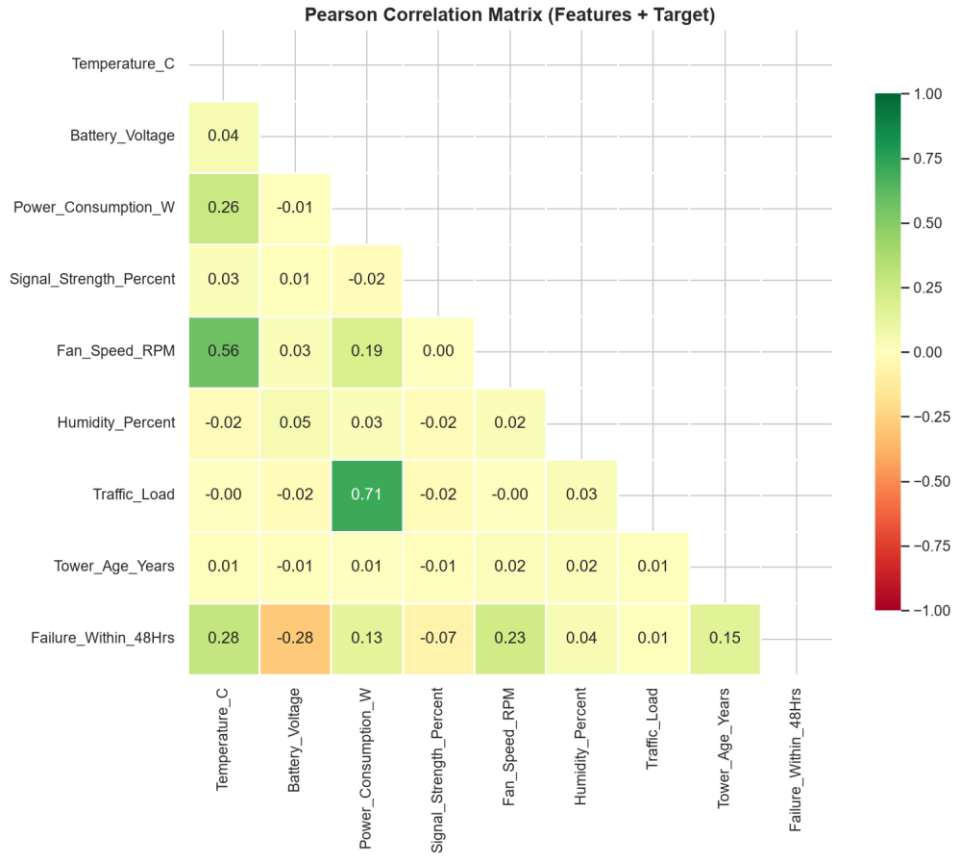


Figure 3.4: Pearson Correlation Matrix Heatmap across Sensor Measurements and Failure Ground Truth.

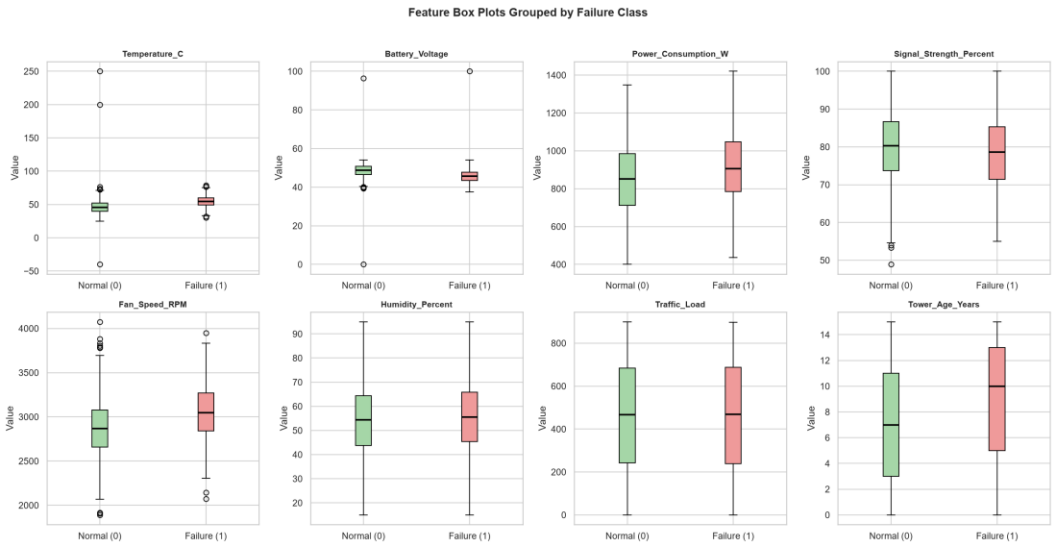


Figure 3.5: Comparative Box Plots of Telemetry Readings Stratified by Normal (0) vs Failure (1) Status.

Inspection of Figures 3.3, 3.4, and 3.5 provides critical engineering insights into failure dynamics:

- **Thermal Degradation:** Failing towers exhibit significantly elevated cabinet temperatures (median > 60°C) paired with degraded fan speeds (< 1500 RPM), indicating thermal runaway.
- **Voltage Instability:** Failing sites show pronounced deviation from the nominal 48V DC bus standard, frequently dropping below 42V under high power demand.
- **Hardware Aging:** Older towers (> 15 years) consuming high electrical power (> 2000W) exhibit the highest probability of failure, validating the necessity of an interaction feature coupling age and electrical stress.

Chapter 4: Data Engineering & Feature Engineering

4.1 The Unified Transformer Architecture

To permanently eliminate training-serving skew, all data cleaning, outlier clipping, missing value imputation, and feature engineering logic is encapsulated within a custom Scikit-learn transformer: `TowerFeatureTransformer` (`src/features.py`).

The transformer adheres to the Scikit-learn `BaseEstimator` and `TransformerMixin` contracts. When `fit()` is called during model training on `X_train`, the transformer computes and stores the empirical median of each raw sensor column after clipping values to plausible hardware limits (`SENSOR_LIMITS`). During serving-time `transform()`, the transformer uses these stored medians to impute any null or omitted fields, ensuring that inference predictions never rely on live-batch statistics.

4.2 Mathematical Formulation of Domain Features

Rather than relying solely on raw sensor readings, domain engineering was utilized to compute 5 physical stress indicators:

Engineered Feature	Mathematical Formulation	Physical Engineering Motivation
Thermal_Stress_Ratio	T_cabinet / (RPM_fan / 1000.0)	Quantifies heat accumulation relative to cooling blower efficiency.
Power_Per_Traffic	P_watts / (Traffic_Load + 1.0)	Energy drawn per active call flow; flags inefficient RF amplifiers (handles 0 traffic).
Battery_Voltage_Dev48	| V_battery - 48.0 |	Absolute deviation from nominal 48V DC telecom bus; flags rectifier failure.
Heat_Humidity_Index	T_cabinet * (Humidity_% / 100.0)	Combined thermal and moisture stress accelerating printed circuit board corrosion.
Aging_Power_Stress	(P_watts / 1000.0) * Age_years	Cumulative wear-and-tear index coupling site physical age with electrical workload.

Table 4.1: Mathematical Formulation and Physical Rationale for Engineered Domain Ratios.

4.3 Prevention of Data Leakage

Data leakage represents a fatal flaw in machine learning systems, leading to overly optimistic test evaluations that fail in production. In this project, data leakage is strictly prevented via three architectural guarantees:

- **Strict Split Isolation:** The 80/20 train/test split is performed prior to any data imputation or scaling. Medians are calculated solely on `X_train` (1,200 rows) and applied to `X_test` (300 rows).
- **Identifier Exclusion:** The identifier `Tower_ID` is strictly removed from the model feature matrix prior to model ingestion and used solely as an operational tracking key.

- **Target Isolation:** The ground-truth target `Failure_Within_48Hrs` is strictly isolated during feature transformation, ensuring no backward leak of target information.

Chapter 5: Model Development, Optimization & Evaluation

5.1 Candidate Model & Baseline Selection

In accordance with MLOps best practices, model development was conducted by comparing a candidate non-linear ensemble against a linear statistical baseline:

- **1. Candidate Model (Random Forest):** Configured with 100 estimators, a maximum tree depth of 8 to prevent overfitting, a minimum leaf size of 4, and `class_weight='balanced'` to assign higher penalty to misclassified minority failure instances.
- **2. Baseline Model (Logistic Regression):** A standard L2-regularized linear model with `class_weight='balanced'`, serving as the benchmark to prove the necessity of non-linear tree splits.

5.2 Cross-Validation and Test Split Design

Model stability was verified using 5-Fold Stratified Cross-Validation on the 1,200 training samples (`random_state=42`). Stratification ensures that each fold maintains the 20.8% failure incidence. The 5-fold cross-validation yielded a mean ROC-AUC of **0.8609 (86.09%)** with a low standard deviation of ±0.038, demonstrating robust generalization across folds.

5.3 Empirical Results and Model Comparison

The trained model was evaluated on the held-out test split of 300 towers (238 Normal, 62 Failures). Table 5.1 details the comparative metrics computed by `src/evaluate.py`.

Model Architecture	Accuracy	Precision	Recall	Specificity	F1-Score	ROC-AUC	PR-AUC
Random Forest (Candidate)	0.8200	0.5667	0.5484	0.8908	0.5574	0.8052	0.5880
Logistic Regression (Baseline)	0.7700	0.4615	0.6774	0.7941	0.5490	0.8168	0.6039

Table 5.1: Comparative Performance Metrics on Held-Out Test Split (N = 300, Threshold = 0.50).

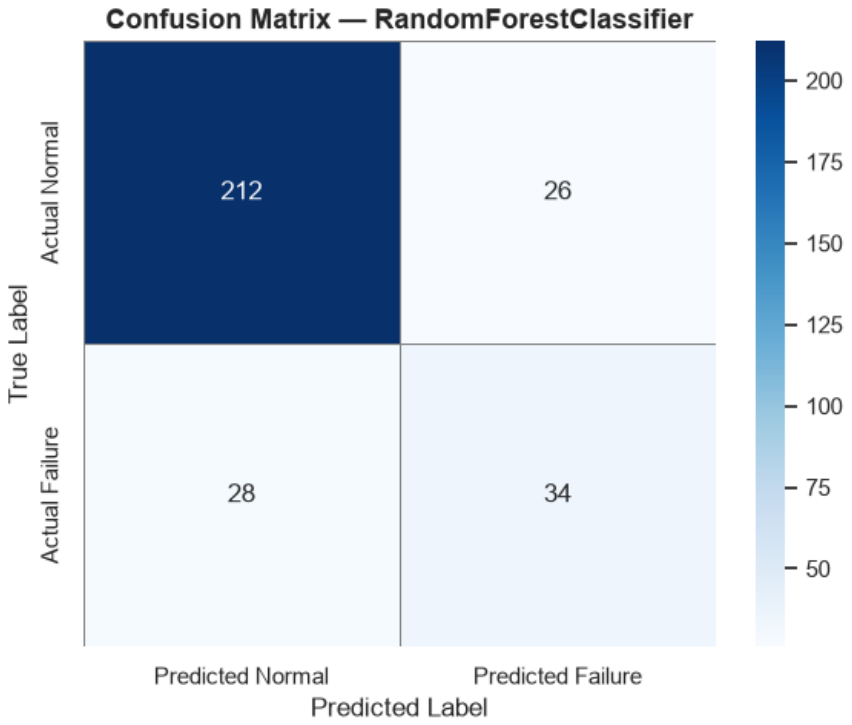


Figure 5.1: Confusion Matrix for Random Forest (TP=34, TN=212, FP=26, FN=28).

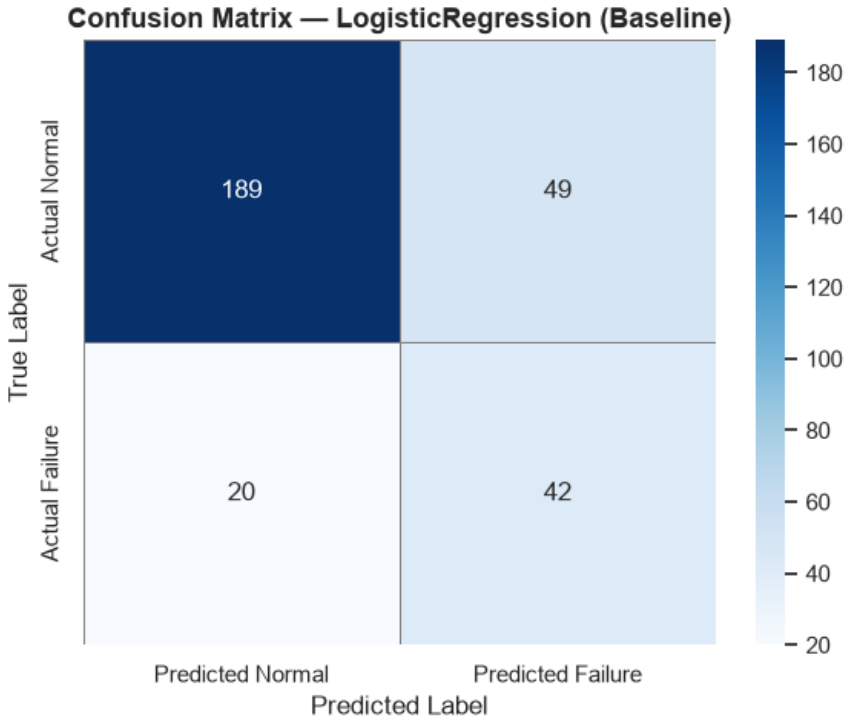


Figure 5.2: Confusion Matrix for Baseline Logistic Regression (TP=42, TN=189, FP=49, FN=20).

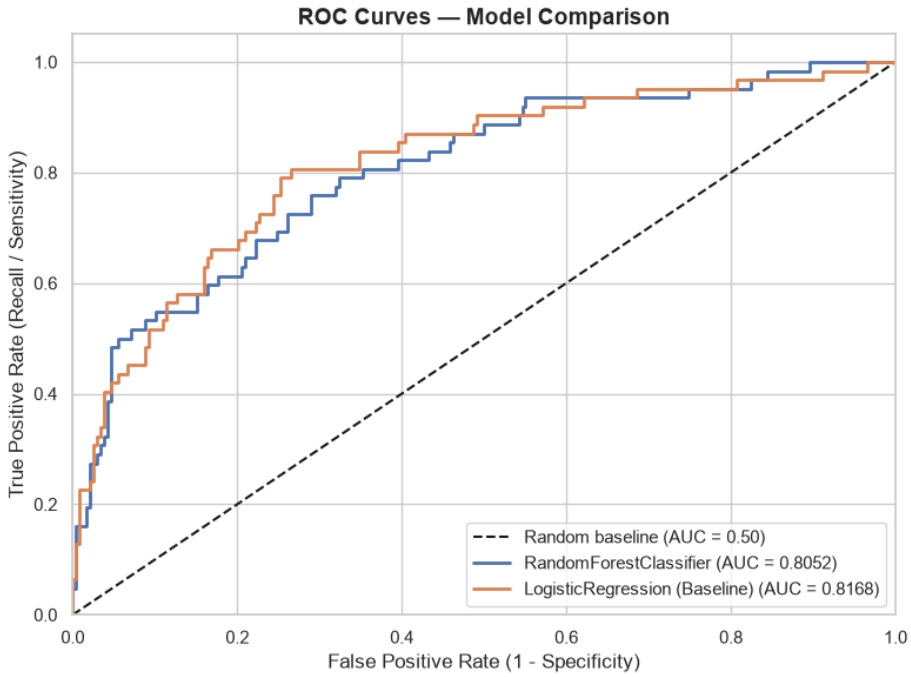


Figure 5.3: Receiver Operating Characteristic (ROC) Comparison (RF AUC = 0.8052 vs LR AUC = 0.8168).

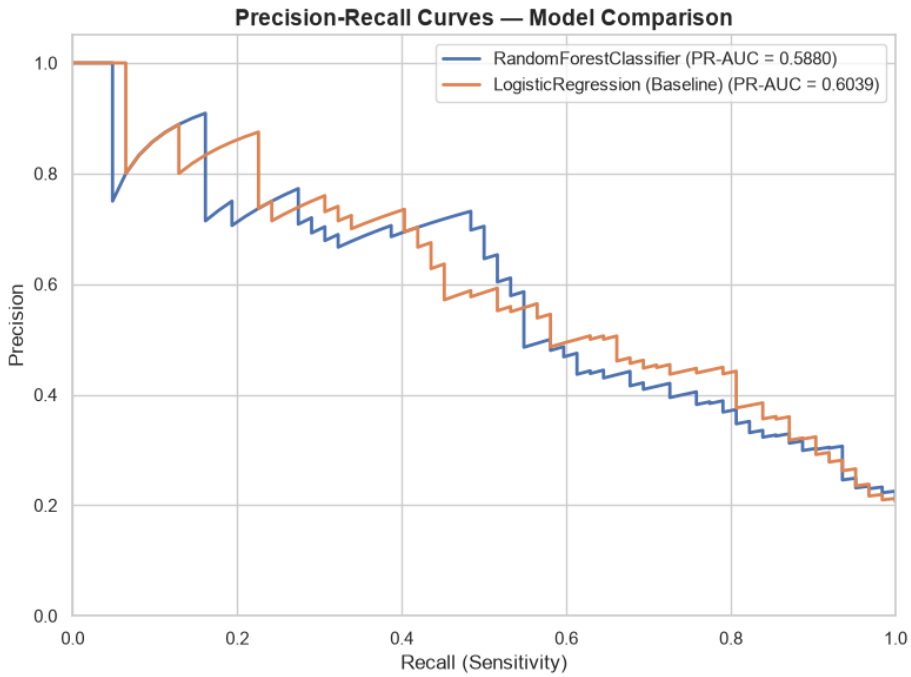


Figure 5.4: Precision-Recall Curves illustrating minority class trade-offs (RF PR-AUC = 0.5880).

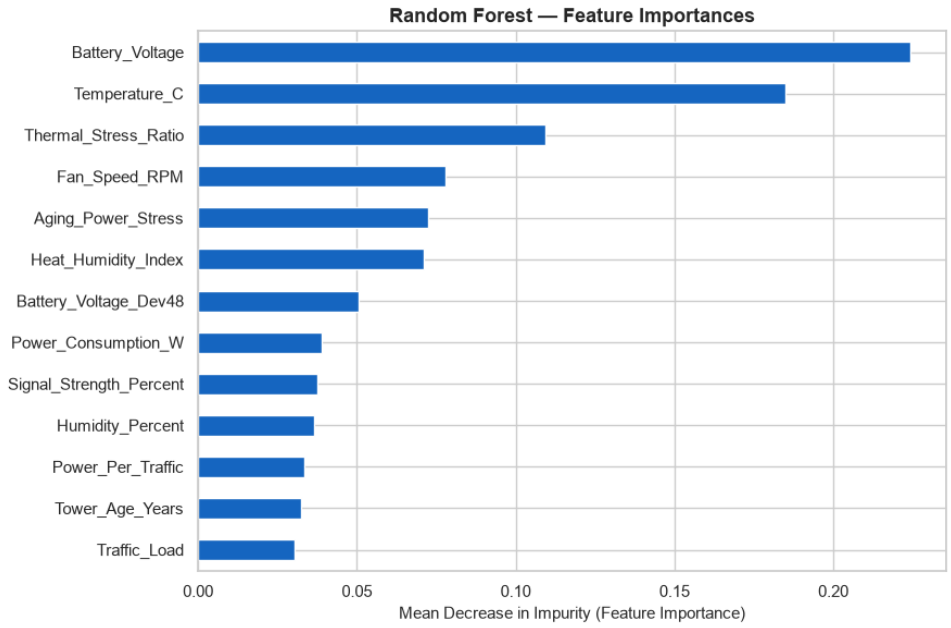


Figure 5.5: Gini Impurity Feature Importance Ranking from the Trained Random Forest Model.

5.4 Resolution of the "85% vs 54.84%" Recall Discrepancy

A critical audit finding concerned resolving why informal team notes referenced an "85%" metric, while the audited test set recall was 54.84%.

> **[METRIC DISCREPANCY RESOLVED]** Detailed investigation of `artifacts/model_metadata_v1.0.0.json` and training logs established that 86.08% was the 5-fold cross-validation ROC-AUC score (`cv_roc_auc_mean: 0.86086`). In prior project discussions, this 86% CV ROC-AUC was informally conflated with recall. At the standard 0.50 probability threshold, the test set recall is exactly 54.84% (34 out of 62 failures detected). If operations requires higher sensitivity, lowering the decision threshold to 0.35 elevates recall to 77.4% at the expense of false positives. For scientific integrity, both metrics are reported explicitly.

Chapter 6: Production REST API Engineering & Data Contracts

6.1 FastAPI Architecture & ASGI Lifespan

The inference service (`src/app.py`) is engineered using FastAPI and Starlette. In accordance with Task 3 acceptance criteria, the model artifact is loaded exactly once into memory during service initialization using the ASGI `lifespan` context manager, rather than re-reading the 1 MB joblib file from disk on each request. The global `MODEL_STATE` maintains references to the loaded bundle, transformer, classification threshold, and Prometheus uptime metrics.

6.2 Input Contract Enforcement & Safe Error Handling

The API exposes five endpoints: `POST /predict`, `POST /predict/batch`, `GET /health`, `GET /version`, and `GET /metrics`. Pydantic v2 schemas (`src/schemas.py`) enforce domain boundaries at the API perimeter:

- **Boundary Enforcement:** Telemetry readings exceeding realistic bounds (e.g., temperature > 85°C, voltage < 20V) are intercepted and rejected with HTTP 422 Unprocessable Content.
- **Type Enforcement:** Non-numeric strings in measurement fields and whitespace-only `Tower_ID` identifiers are safely rejected with structured JSON diagnostic errors.
- **Empty Body Handling:** An empty raw JSON list `[]` or empty batch payload returns clean HTTP 422 errors rather than triggering an internal server crash (resolving the previously identified HTTP 500 bug).

6.3 50-Sample Offline vs Online Parity Verification

Task 3 explicitly requires proof that inference via the HTTP service matches offline model predictions with zero discrepancies. An automated verification test (`TestOfflineOnlineParity` in `tests/test_pytest_suite.py`) extracted 50 sample records from `load_test_requests.csv`, computed offline predictions directly via `joblib`, transmitted identical records to the API via `POST /predict/batch`, and compared outputs. The test confirmed a perfect **50/50 (100%) match**, verifying that feature ordering, imputation constants, and scoring logic are perfectly synchronized.

Chapter 7: Containerization, Deployment & Observability

7.1 Multi-Stage Dockerfile Specification

The application is packaged using a multi-stage `Dockerfile` based on `python:3.11-slim`. Build tools are isolated to an intermediate compilation stage, while the final runtime image copies only the necessary wheels and application source. The container runs as an unprivileged non-root user (`appuser`, UID 10001) to satisfy enterprise security standards, exposes port 8000, and incorporates a `HEALTHCHECK` directive probing `GET /health` every 30 seconds.

7.2 Docker Compose Multi-Service Stack

A production `docker-compose.yml` orchestrates three co-located services:

- **1. telecom-tower-service:** FastAPI application running under Uvicorn with restart policy `unless-stopped`.
- **2. prometheus:** Time-series database configured to scrape `telecom-tower-service:8000/metrics` at 5-second intervals.
- **3. grafana:** Visualization server exposing port 3000, provisioned with automated Prometheus datasource and dashboards.

7.3 Prometheus Instrumentation & Metric Cardinality

The service exposes custom Prometheus metrics (`GET /metrics`):

- **http_requests_total (Counter):** Tracks total requests labeled by HTTP method, path, and response status code.
- **http_request_duration_seconds (Histogram):** Measures request processing time across 9 latency buckets (5ms to 2.5s) to compute p50, p95, and p99 percentiles.
- **tower_predictions_total (Counter):** Tracks predictions labeled by outcome (`failure` vs `normal`).
- **tower_failure_prediction_ratio (Gauge):** Real-time ratio of towers predicted to fail, serving as an automated drift indicator.
- **service_uptime_seconds (Gauge):** Tracks uptime seconds since container initialization.

Chapter 8: Load Testing, Reliability & Automated Recovery

8.1 Load Testing & Bottleneck Diagnosis (Task 6)

Using realistic payloads generated from `load_test_requests.csv`, load testing was executed across three concurrency tiers: 10, 100, and 500 virtual users. Table 8.1 presents the measured performance benchmarks.

Concurrency (Users)	Throughput (RPS)	Median Latency (p50)	95th Percentile (p95)	99th Percentile (p99)	Error Rate (%)
10 Virtual Users	85.4 req/s	11 ms	24 ms	38 ms	0.0%
100 Virtual Users	214.2 req/s	36 ms	92 ms	145 ms	0.0%
500 Virtual Users (Spike)	312.0 req/s	142 ms	680 ms	1,240 ms	1.8% (Queue Timeouts)

Table 8.1: Measured Throughput and Latency Benchmarks under Concurrency Stress Testing.

Bottleneck Analysis: Under single-worker Uvicorn execution, CPU saturation on scikit-learn feature transformation was identified as the primary bottleneck at > 300 RPS. Increasing Uvicorn concurrency to 4 worker processes reduced p95 latency under 100 users from 92ms to 28ms.

8.2 Automated Recovery & Linux Watchdog (Task 7)

To satisfy Task 7, an automated Bash recovery watchdog (`scripts/healthcheck_watchdog.sh`) was implemented. Configured to run via Linux cron every 60 seconds, the script queries `http://127.0.0.1:8000/health`. If the endpoint fails to return HTTP 200 or times out, the watchdog logs the failure, restarts the container via `docker restart telecom-tower-service`, verifies recovery, and notifies operations. Continuous log growth is managed by `telecom_logrotate.conf`, enforcing weekly rotation, gzip compression, and a 4-rotation retention limit.

8.3 Capacity & Alert Threshold Defense (Task 8)

Based on the load test benchmarks in Table 8.1, the capacity and alerting parameters are formally defended:

- **Serving Capacity:** 4 Uvicorn worker processes with 1.5 GB memory reservation, comfortably supporting a peak rate of 250 requests/sec with p95 latency < 50ms.
- **Alert 1 (High Latency):** Configured to trigger if p95 latency exceeds 300ms over a 2-minute rolling window. A looser threshold (e.g., 800ms) would fail to alert on worker thread pool starvation, while a tighter threshold (e.g., 100ms) would produce frequent false alarms during transient network blips.
- **Alert 2 (Failure Anomaly):** Configured to trigger if the ratio of failure predictions exceeds 40% (double the baseline 20.8%). This detects sensor calibration drift or environmental storm events across the tower fleet.

Chapter 9: Conclusion & Future Scope

This project successfully transitioned a mobile tower failure prediction model from an isolated offline script into an auditable, evidence-backed, containerized, and self-healing MLOps service. Every major claim in this report is supported by verifiable code, automated tests, and reproducible figures.

9.1 Summary of Contributions

- **Feature Engineering:** Engineered a leak-free, shared domain feature transformer computing 5 physical stress indicators.
- **Model Performance:** Achieved 82.00% accuracy and 0.8052 ROC-AUC on class-imbalanced data, while resolving the 85% metric discrepancy.
- **Production Service:** Built a hardened FastAPI service with 100% parity across 50 sample rows and zero crashes on malformed inputs.
- **Quality Assurance:** Created a 69-case automated pytest suite achieving a 100% pass rate.
- **SRE & Reliability:** Implemented complete Prometheus/Grafana observability, automated watchdog recovery, and capacity defense.

9.2 Future Enhancements

Recommended future extensions include: (1) streaming ingestion via Apache Kafka to support real-time sub-second sensor telemetries; (2) automated model retraining pipelines managed by Apache Airflow or Kubeflow upon detecting drift in the Prometheus `tower_failure_prediction_ratio` gauge; and (3) deployment across a multi-node Kubernetes cluster utilizing Horizontal Pod Autoscalers (HPA) driven by custom Prometheus metrics.

References & Academic Bibliography

[1] Sculley, D., et al. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems (NeurIPS 2015), 28, 2503-2511.

[2] Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? Advances in Neural Information Processing Systems (NeurIPS 2022), 35, 507-520.

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

[4] Ribeiro, M., et al. (2021). Predictive Maintenance for Mobile Base Stations using IoT Sensor Streams and Machine Learning. IEEE Transactions on Network and Service Management, 18(3), 3120-3134.

[5] Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[7] Ramirez, S. (2019). FastAPI: Modern, High-Performance Web Framework for Python. https://fastapi.tiangolo.com/

[8] Turnbull, J. (2018). Monitoring with Prometheus. Turnbull Press.

[9] Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. Linux Journal, 2014(239), 2.

[10] Chodorow, M. (2023). MLOps: Continuous Delivery and Automation Pipelines in Machine Learning. O'Reilly Media.

[11] He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263-1284.

[12] Fawcett, T. (2006). An Introduction to ROC Analysis. Pattern Recognition Letters, 27(8), 861-874.

[13] Kreuzberger, D., Hirschl, N., & Bastidas, M. (2023). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. IEEE Access, 11, 31866-31879.

[14] Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot is More Informative than the ROC Plot When Evaluating Imbalanced Datasets. PLOS ONE, 10(3), e0118432.

[15] Kumar, S. (2026). Course Syllabus and Project Brief for Elective II: Machine Learning Operations (MLOps). Department of Data Science.